The PHINIX+ System Architecture Documentation

Part 1: The CPU

Version 1.0 (October Nth 2023)

by Martin Andronikos

Contents

1 Introduction 3
1.1 Glossary 3
1.2 Meta Details 3
2 Registers 4
2.1 General-Purpose Registers 4
2.2 Special-Purpose Registers 5
2.3 The Standard Register Convention 5
3 Instruction Formats 6
3.1 Instruction Field Nomenclature 6
3.2 Instruction Format Nomenclature 7
3.3 Operand Types 7

1 Introduction

This document is the official specification for the architecture of the PHINIX+ Central Processing Unit. It is intended to explain in detail the capabilities and the layout of the processor in an abstract manner in order to remain agnostic of the possible implementations of it.

While this document doesn't try to make any assertions of a "correct" sort of implementation, the architecture was built with the intention to exploit pipelining to gain in performance. As a consequence of this design foresight, pipelining is partly the namesake of the processor. Specifically, PHINIX+ stands for Pipelined High-speed INteger Instruction eXecutor. The + is due to this design being an overhauled version of a previous system which never went much beyond simulation.

In short, the PHINIX+ processor is inspired by many existing architectures yet it adapts on the features borrowed and also adds some unique elements. Specifically, it borrows from many RISC architectures including RISC-V and MIPS, but it also has features seen mostly on CISC processors like x86 and M68k. It is best described as a 16-bit load-store variable-length 2-op instructions architecture with dual register files and both flagged and flagless condition codes, operating on a unified word-addressed code+data memory space, often referred to as a Von Neumann architecture. A rudimentary system for interrupts, a memory management system and a privileged mode are also available.

PHINIX+ is open source hardware and as such the source for this documentation and for reference implementations are available under GPLv3 over at the PHINIX+ GitHub organization.

1.1 Glossary

Esoteric terminology or PHINIX+-specific definitions of are listed below:

- **Architecture**: The abstract definition of instructions and registers present in the design.
- **Implementation**: The realisation of the architecture in hardware or software following a defined execution model.
- **Immediate**: A numeric value embedded directly in an instruction.
- **Register**: A small unit of memory inside the processor that is specified by the architecture.
- **Register file**: A grouping of registers with similar purposes that acts as an addressable unit.
- **Word**: One unit of memory in the PHINIX+ architecture, equal to 16 bits.

1.2 Meta Details

This document is my foray into Typst and document preparation in general. If you find any mistakes or want to provide feedback please visit the project's discord server: PHINIX+ Forum. Additionally, I can not guarantee the same level of formality displayed in other documentation sheets due to my relative unfamiliarity with the subject. I hope you understand and won't let that aspect detract from the value of this document.

Styling decisions for this document are the following:

- Normal text is styled using the *Free Serif* typeface.
- Headings are styled using the *DejaVu Sans* typeface in bold. Additionally, top level headings are underlined.
- Internal links (labels) are colored blue without underlining, with the exception of the Contents page.
- External (hyper)links are colored blue and underlined.

2 Registers

The first thing that a programmer learns about an architecture, in general, is what registers are exposed for use. The set of registers that are available is, in a sense, part of the personality of an architecture. That is because registers are, strictly, per the RISC design pattern, the only medium on which calculations can be done. They can be thought of as hardware-provided variables.

PHINIX+'s registers are, at first glance, unconventional though their layout borrows from the M68k which also had two register files, one for data referred to as Dx and one for addresses (pointers) referred to as Ax. In our case, Dx corresponds with the primary register file displayed in Table 1 and Ax with the secondary in Table 2. There also exist a few special-purpose registers, in addition to the two register files, displayed in Table 3, whose functions will be explained on a case-by-case basis.

2.1 General-Purpose Registers

The primary and secondary register files have no restrictions by the hardware on how they are used, with the exception of $x0 which reads always as 0 and discards writes, and of $yC-$yF which are reseved for system software and are unavailable if the system implements and is in user mode. Each instruction can access at most two registers, each from either register file. Writing back to both registers is allowed only when they originated from different register files. In other words, each register file concurrently and independantly allows two reads and one write per instruction. All of the general-purpose registers are 16 bits.

Arch.	Conv.	Description	Save?
$x0	$zr	Constant Zero	None
$x1	$at	Assembler Temporary	Caller
$x2	$rp	Return Pointer	Caller
$x3	$a0	Arg./Return Value #0	Caller
$x4	$a1	Arg./Return Value #1	Caller
$x5	$a2	Arg./Return Value #2	Caller
$x6	$t0	Temporary Value #0	Caller
$x7	$t1	Temporary Value #1	Caller
$x8	$t2	Temporary Value #2	Caller
$x9	$s0	Saved Value #0	Callee
$xA	$s1	Saved Value #1	Callee
$xB	$s2	Saved Value #2	Callee
$xC	$s3	Saved Value #3	Callee
$xD	$s4	Saved Value #4	Callee
$xE	$s5	Saved Value #5	Callee
$xF	$fp	Frame Pointer	Callee

Table 1: PHINIX+'s primary register file

Arch.	Conv.	Description	Save?
$y0	$t3	Temporary Value #3	Caller
$y1	$t4	Temporary Value #4	Caller
$y2	$t5	Temporary Value #5	Caller
$y3	$a3	Arg./Return Value #3	Caller
$y4	$a4	Arg./Return Value #4	Caller
$y5	$a5	Arg./Return Value #5	Caller
$y6	$s6	Saved Value #6	Callee
$y7	$s7	Saved Value #7	Callee
$y8	$s8	Saved Value #8	Callee
$y9	$s9	Saved Value #9	Callee
$yA	$gp	Global Pointer	Callee
$yB	$sp	User Stack Pointer	Callee
$yC	$k0	System Value #0	Callee
$yD	$k1	System Value #1	Callee
$yE	$k2	System Value #2	Callee
$yF	$kp	System Stack Pointer	Callee

Table 2: PHINIX+'s secondary register file

2.2 Special-Purpose Registers

The special-purpose registers, by their name, have more narrow use-cases. Each one serves different functionality and has unique accompanying instructions that operate on it. All of the special-purpose registers are also 16 bits.

Arch.	Description	Save?
$ip	Instruction Pointer	None
$jp	Interrupt Return Pointer	None
$rf	Reference Register	Caller
$st	Status Register	Caller

Table 3: PHINIX+'s special-purpose registers

A description of each of the registers is given here:

- **$ip**: The instruction pointer; a register containing the address of the currently executing instruction.
- **$jp**: The alternate instruction pointer or interrupt return pointer; in which the main one is saved when it's time to service an interrupt. This is required because, when exiting the interrupt handler, none of the other registers may have modified values (due to the interrupt's need to be transparent to user code) which leaves the need for a dedicated register to contain the return address. As a result, this register is also inaccessible from user mode code due to its volatility.
- **$rf**: The reference register; having a dual purpose, can either be used to contain a reference value to compare against using special condition codes, or be used as a vector of 16 1-bit accumulators to do bit math either on condition codes to create more compound ones or on user-provided values. This is one of the most innovative features of PHINIX+.
- **$st**: The status register; contains the four operation flags (NVCZ) generated after most calculations, software interrupt trigger bits, and general-purpose input/output bits intended to be used as status information to the outside world (eg. panel lights).

2.3 The Standard Register Convention

Soon™

3 Instruction Formats

Since data and instructions (the components of code) are free to exist in the same memory space, the instructions have to be aligned to the size of the data which in this case is 16 bits. Some formats fit instructions in just one word, but others require an additional immediate value which takes up a second word. In total, there exist 8 distinct formats to accomodate most instructions' argument needs without having unused operand fields. Table 4 lists those 8 formats that all PHINIX+ instructions fit into.

	[15:12]	[11:8]	[7:4]	[3:0]	Extd. Imm.
S	Opc.[3:0]	Imm.[7:0]		Dst.	✗
MR	Opc.[7:0]		Src.	Dst.	✗
MR+	Opc.[7:0]		Src.	Dst.	✔
MRI	Opc.[7:0]		Imm.[3:0]	Dst.	✗
MI	Opc.[7:0]		Imm.[7:0]		✗
LR	Opc.[11:0]			Dst.	✗
LR+	Opc.[11:0]			Dst.	✔
LI	Opc.[11:0]			Imm.[3:0]	✗

Table 4: PHINIX+'s instruction formats

3.1 Instruction Field Nomenclature

The terms used in Table 4 are explained and expanded upon here:

- **Extd. Imm.** stands for *Extended Immediate*. It is always 16 bits in length. It takes up an entire second word following after the main word containing a number of fields described below. Depending on the instruction, it may not exist and as a result the space is ocupied by the first word of the next instruction. This is the only aspect of the instructions that is variable-length.
- **Opc.** stands for *Operation Code* and is between 4 and 12 bits in length. This field identifies which instruction is actually being executed and dictates whether the Extended Immediate is present and in general which of the 8 instruction formats is to be used.
- **Src.** stands for *Source* and is a register address which is always 4 bits in length. This register is only ever read from during instruction execution. It can indicate a register from either of the two register files, depending on the specific instruction.
- **Dst.** stands for *Destination* and is also a register address. This register is first read from and then written to at the end of an instruction. It can indicate a register from either of the two register files, depending on the specific instruction.
- **Imm.** stands for *Immediate* (value) and is between 4 and 8 bits in length. It differs from the Extended Immediate in that it doesn't ocupy a second word but rather uses space available within the first word. 4-bit immediates are referred to as *Short* and 8-bit immediates as *Long*.

Additionally, the two numbers in square brackets separated by a colon notation signifies a range of bits within the word. For example, the notation [3:0] signifies the first 4 bits (bit indecies starting at 0). This notation is part of the Verilog Hardware Description Language and is used here for parity.

3.2 Instruction Format Nomenclature

Each instruction format has its own name, as seen in column 1 of Table 4. Those names are not given randomly but are instead based on the layout of the format itself. There are three parts that come together to form a name. The first indicates the size of the operation code using the letters S, M, and L for *Small* (4-bit), *Medium* (8-bit), and *Large* (12-bit) respectively. The second part indicates the type of operands present. There are thee variants, R, I, and RI. R stands for *Register* and I stands for *Immediate*. Lastly, the third part indicates the presence of a second word in the instruction using the symbol +.

It is important to point out that for small operation code formats, 12 bits are available for operands, for medium operation code formats 8 bits are available, and for long operation code formats only 4 bits are available. This restriction affects how the formats are named due to the inherent restrictions of each format.

Now that each component is explained, each format's name will be broken down individually:

- **S**: No other layout information needed for this one since it's the only one with a small operation code.
- **MR**: Medium operation code with two registers as operands.
- **MR+**: Same as the above but extended with an immediate value word (16 bits).
- **MRI**: Medium operation code with one register and one immediate (4 bits) as operands.
- **MI**: Medium operation code with one immediate (8 bits) as operand.
- **LR**: Long operation code with one register as operand.
- **LR+**: Same as the above but extended with an immediate value word (16 bits).
- **LI**: Long operation code with one immediate (4 bits) as operand.

3.3 Operand Types

As it may be apparent by this point, instructions, broadly, have two options for their operands. They can either take the value off of a register after addressing a register file or embed the needed value directly into the instruction using immediates of varying sizes. But that only covers the explicit operands. There are certain instructions that use implicit operands such as constant values or registers outside of a register file for which there exists no address and thus its access must be indicated by the operation code itself. Furthermore, each can be used for a variety of ways, or combined together to form more complex behaviours. The ways each type of operand is used will be listed below starting with individual applications and then following with combined ones.

The four different types of explicit operands are thus the following:

- **reg**: A 4-bit register address.
- **imm4**: A 4-bit immediate value.
- **imm8**: An 8-bit immediate value.
- **imm16**: A 16-bit immediate value.